

Transações

definição

Conjunto de operações que ou são **TODAS** confirmadas ou são **TODAS** abortadas

↳ conjunto de selects executadas por um SGBD.

↳ ela processa itens de dados: pode ser um arquivo, um registro, uma coluna, um bloco...

depende da granularidade (tam do item) do SGBD

ex: se pessoa A quer mexer na coluna sexo de cliente e pessoa B quer mexer no salário, se tivermos granularidade de colunas pode ser ao mesmo tempo. Se for em blocos, não

quanto + fina a granularidade, maior a concorrência, porém + itens p/controlar

Maior concorrência → executar + transações em uma CPU compartilhada.

Controle de concorrência → uma interação de um usuário não pode interferir na outra (isolado)

Problemas de concorrência

Atualização perdida → sobrescrever itens
Leitura suja → transação aborta e item é inválido
Leitura não repetitiva → lê valores diferentes
Resumo incorreto → combinação dos 3 acima, se fizermos uma agregação pode ser incorreto

esses são os problemas que podem existir caso tenhamos uma execução paralela

↓ concorrência
↓ performance

Motivos de abortar

Falhas de sistema → hardware, software ou rede

Operação → lógica de programação errada

concorrência → controle por ler itens sujos

condição de execução → ir saldo negativo por ex

Disco → blocos perdidos

catástrofe → formatação disco, blackout, incêndio, roubo

Todo SGBD tem mecanismos para lidar com falhas

Logging → histórico de transações

Dumping → cópia de segurança do BD → dentro do comp. → corremp

Backup → externo ao comp. → servidor morreu, ex: queimou

Níveis diferentes de segurança

Escalonamento → fatorial de n (n!)

Intercalação das operações a serem feitas

a → abort

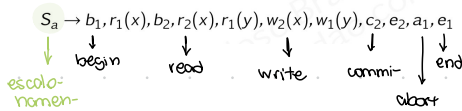
c → commit

b → begin

e → end

r → read

w → write



Begin e end são óbvios, normalmente não colocamos no escalonamento

Completo → possui todas as operações de cada transação APENAS e ordem é preservada

$T_1 \rightarrow r(x), r(y), w(y), a$
 $T_2 \rightarrow r(x), w(x), c$

$S \rightarrow r_1(x), r_2(x), r_1(y), w_2(x), w_1(y), c_2, a_1$

todos estão presentes ✓
estão em ordem ✓
conflito estão em ordem ✓

Incompleto →

Se for:

$T_1 \rightarrow r(x), r(y), w(y), a$

$T_2 \rightarrow r(y), w(x), c$

$S \rightarrow r_1(x), r_2(y), r_1(y), w_2(x), w_1(y), c_2, a_1$

conflito em outra ordem não é completo

Completo garante atomicidade e consistência
Recuperável garante durabilidade

recuperável

▶ LEITURA SUJA → pode demandar reversão de confirmação

$S_a \rightarrow r_1(x), r_2(x), w_1(x), r_1(y), w_2(x), r_1(x), c_1, c_2$

▶ Nenhuma transação T que leia um item escrito por outra transação T' pode confirmar antes de T'

$S_a \rightarrow r_1(x), r_2(x), w_1(x), r_1(y), w_2(x), r_1(x), c_2, c_1$

transação que está lendo confirma antes da que escreveu

leitura suja

→ escalonamento não é recuperável

conflitos

→ operam sob mesmo item

→ pelo menos uma é write

→ são transações diferentes

isolamento e concorrência

SERIAL → sem intercalação, com todas as operações de uma transação precedendo todas as operações de outra

$S_a \rightarrow r_1(x), w_1(x), r_1(y), r_1(x), c_1, r_2(x), w_2(x), c_2$

▶ ISOLAMENTO → transação não sofre interferência de outra

▶ CONCORRÊNCIA → limitada, gerando ociosidade de CPU

NÃO SERIAL → operações de transações intercaladas

$S_a \rightarrow r_1(x), r_2(x), w_1(x), r_1(y), w_2(x), r_1(x), c_2, c_1$

▶ ISOLAMENTO → não é garantido

▶ CONCORRÊNCIA → melhor aproveitamento de CPU

Se vir ciclo, NÃO É SERIALIZÁVEL

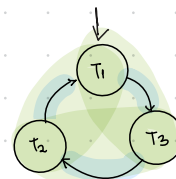
não garante isolamento

Queremos um escalonamento que tenha concorrência mas garanta que não interfira

serializável

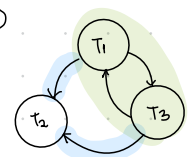
(determinar esc. não serializáveis que se comportam como serializáveis)

grafo de precedência



Completo pois não há ciclos entre duas transações

Não serializável pois há ciclo de forma geral



Incompleto pois há ciclos entre duas transações

Se realizável pois não há ciclo de forma geral